

# Implement Client Script & UI Policy (Incident)

## 3. Solution Design

### Design Overview

The solution combines one UI Policy with UI Policy Actions and three Client Scripts: onChange, onSubmit, and onCellEdit.

### UI Policy – High Impact Control

- Name: High Impact Control.
- Table: Incident.
- Active: true.
- Condition: Impact is 1 – High.
- Assignment group: mandatory.
- Reverse if false: true.

### UI Policy Action – Urgency

- Field: Urgency.
- Read-only: true.
- Visible: unchanged.
- Effect: when Impact is High, Urgency cannot be changed by the user.

### onChange Client Script

Name: Auto set urgency for high impact. Table: Incident. Type: onChange. Field: Impact. Active: true.

Script:

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == "") {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

### onSubmit Client Script

Name: Prevent save if Assigned To missing. Table: Incident. Type: onSubmit. Active: true.

Script:

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == "") {
```

```
g_form.showErrorBox(  
  'assigned_to',  
  'Assigned To is mandatory for High impact incidents.'  
);  
return false;  
}  
return true;  
}
```

### **onCellEdit Client Script**

Name: Prevent state change via list edit. Table: Incident. Type: onCellEdit. Field: State. Active: true.

Script:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
  alert('State cannot be updated using list editing. Please open the Incident.');
```

```
  callback(false);  
}
```

Source basis: supplied 18-page Project 3 PDF. Unsupported project metadata, screenshots, execution results, and dates are not fabricated.